

DEVIXO SOLUTIONS

AI/ML INTERNSHIP PROGRAM

TASK: BUILD AND COMPARE MACHINE
LEARNING MODELS

PROJECT NAME: EMPLOYEE SALARY
PREDICTION

NAME: AIMA SHAHID

INTERNSHIP DOMAIN: AI/ML

SUBMITTED DATE: 30-AUGUST-2026

1-PROJECT OBJECTIVE:

The objective of this project is to build and compare different machine learning regression models for predicting employee salaries. The project uses data preprocessing techniques such as handling missing values, removing duplicate records, encoding categorical features, and scaling numerical features. Three machine learning algorithms-Linear Regression, Decision Tree Regressor, and Random Forest Regressor are trained and evaluated using regression performance metrics.

2-PROBLEM STATEMENT:

Employee salary depends on different factors such as age, gender, department, experience, and performance. The aim of this project is to use these employee features to predict salary using machine learning. Different regression models are developed and compared to identify the best-performing model.

3-TECHNOLOGIES USED:

- Python
- Jupiter Notebook
- Pandas
- NumPy
- Scikit-learn
- Matplotlib
- Seaborn

4-Dataset Description:

The dataset contains employee-related information used for salary prediction. It includes features such as Employee ID, Age, Gender, Department, Experience Years, and Performance Score. The target variable is Salary. The dataset is used to develop a regression model that predicts employee salary based on the available features.

- **Problem Type:** Regression

- **Target Variable:** Salary
- **Training Records:** 800
- **Testing Records:** 200
- **Total Records Used:** 1000
- **Input Features:** 9

5-DATA PREPROCESSING:

- **Missing Values:** Missing values in Experience_Years and Performance_Score were handled using the median of the respective columns.
- **Duplicate Records:** 20 duplicate records were identified and removed.
- **Categorical Encoding:** The categorical features Gender and Department were converted into numerical values using one-hot encoding.
- **Feature Scaling:** Numerical features were scaled using StandardScaler.
- **Train-Test Split:** The dataset was divided into 80% training data and 20% testing data.

6-MACHINE LEARNING MODELS:

1.Linear Regression:

Linear Regression is a simple regression algorithm that predicts the target variable by finding a linear relationship between the input features and salary.

2. Decision Tree Regressor:

Decision Tree Regressor predicts salary by creating decision rules based on the input features.

3. Random Forest Regressor:

Random Forest Regressor combines multiple decision trees to make predictions and improve the overall performance.

7-MODEL EVALUATION:

The trained models were evaluated using four regression metrics: Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and R^2 Score. Lower values of MAE, MSE, and RMSE indicate lower prediction errors, while a higher R^2 Score indicates better model performance.

Model	MAE	MSE	RMSE	R ² Score
Linear Regression	5928.79	57656979.67	7593.219	0.721
Decision Tree	14273.80	2425817245.71	49252.5862	-10.75
Random Forest	7024.23	183504840.266	13546.396	0.111

Linear Regression performed the best with the lowest errors and highest R² Score (0.721). Random Forest performed second, while Decision Tree performed the worst. Therefore, Linear Regression is the recommended model for this dataset.

8-MODEL COMPARISON:

The three models were compared using their evaluation metrics. Linear Regression showed the best overall performance, while Random Forest performed moderately and Decision Tree performed the worst.

Model	MAE	R ² Score	Performance
Linear Regression	5928.79	0.721	Best
Random Forest	7024.23	0.111	Second
Decision Tree	14273.80	-10.75	Worst

9-Code Screenshot and Explanation:

1. Data Preprocessing:

```
#Load the Dataset
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sn
import sklearn
```

```
data=pd.read_csv("../dataset/employee_performance.csv")
```

```
data.head()
```

	Employee_ID	Age	Gender	Department	Experience_Years	Performance_Score	Salary
0	E0524	49	Female	Operations	5.6	67.8	33013.0
1	E0603	55	Female	Sales	5.5	NaN	42640.0
2	E0527	29	Female	Sales	10.7	68.8	72180.0
3	E0032	30	Female	Operations	9.8	NaN	65750.0
4	E0617	58	Male	Sales	11.5	72.8	74830.0

Explanation:

The dataset was loaded using Pandas, and the first five rows were displayed using `data.head()` to verify that the data was loaded correctly.

2. Categorical Features Encoding:

```
# Encode Categorical Features
data.select_dtypes(include='object').columns
```

```
Index(['Employee_ID', 'Gender', 'Department'], dtype='object')
```

```
data=pd.get_dummies(data,
                    columns=['Gender', 'Department'],
                    drop_first=True,
                    dtype=int)
```

```
data.head()
```

	Employee_ID	Age	Experience_Years	Performance_Score	Salary	Gender_Male	Department_HR	Department_IT	Department_Marketing	Department_Operations	D
0	E0524	49	5.6	67.80	33013.0	0	0	0	0	1	
1	E0603	55	5.5	70.05	42640.0	0	0	0	0	0	
2	E0527	29	10.7	68.80	72180.0	0	0	0	0	0	
3	E0032	30	9.8	70.05	65750.0	0	0	0	0	1	
4	E0617	58	11.5	72.80	74830.0	1	0	0	0	0	

Explanation:

Categorical features such as Gender and Department were converted into numerical values using one-hot encoding so they could be used by machine learning models.

3. Train-Test Split:

```
from sklearn.model_selection import train_test_split
X= data.drop(['Employee_ID','Salary'],axis=1)
Y=data['Salary']
X_train,X_test,Y_train,Y_test=train_test_split(
    X,Y,test_size=0.2,
    random_state=42)
```

```
print(X_train.shape)
```

```
(800, 9)
```

```
print(X_test.shape)
```

```
(200, 9)
```

```
print(Y_train.shape)
```

```
(800,)
```

```
print(Y_test.shape)
```

```
(200,)
```

Explanation:

The dataset was divided into 80% training data and 20% testing data. The training data was used to train the machine learning models, while the testing data was used to evaluate their performance.

4. Model Development:

```
# Linear Regression
from sklearn.linear_model import LinearRegression
model_lr=LinearRegression()
model_lr.fit(X_train,Y_train)
```

LinearRegression ⓘ ?

► Parameters

```
# Decision Tree Regressor
from sklearn.tree import DecisionTreeRegressor
model_dt=DecisionTreeRegressor(random_state=42)
model_dt.fit(X_train,Y_train)
```

DecisionTreeRegressor ⓘ ?

► Parameters

```
# Random Forest Regressor
from sklearn.ensemble import RandomForestRegressor
model_rf=RandomForestRegressor(random_state=42)
model_rf.fit(X_train,Y_train)
```

RandomForestRegressor ⓘ ?

► Parameters

Explanation:

In this phase, three regression algorithms—Linear Regression (baseline model assuming a linear relationship), Decision Tree Regressor (capturing non-linear patterns through decision splits), and Random Forest Regressor (an ensemble method combining multiple trees to enhance accuracy and reduce overfitting)—were initialized and trained on the training dataset (X_train, Y_train), with random_state=42 used where applicable to ensure reproducible results.

10. ADVANTAGES & LIMITATIONS:

Model	Advantage	Limitation
Linear Regression	Simple and fast	Assume a linear relationship
Decision Tree	Easy to understand	Can overfit

Random Forest	Good prediction performance	More complex and slower
---------------	-----------------------------	-------------------------

Explanation:

Each model has different strengths and limitations. Linear Regression is simple and fast, Decision Tree is easy to understand, while Random Forest can provide good performance but is more complex.

11-CONCLUSION:

The project successfully developed and compared three machine learning regression models for employee salary prediction. The data was preprocessed by handling missing values, removing duplicates, encoding categorical features, scaling numerical features, and splitting the dataset into training and testing sets.

Among the three models, Linear Regression performed the best, achieving the lowest prediction errors and the highest R^2 Score of 0.721. Therefore, Linear Regression is recommended as the best-performing model for this dataset.